

TinyOS 安装教程

本教程仿照《TinyOS 安装教程 2024》的分步写法整理，但安装路径不再使用 Ubuntu 14.04 + VMware + apt 安装 tinyos-2.1.2 的旧方案，而改为“新版本 Ubuntu 宿主机 + Docker + 容器内源码编译 nesc 与 tinyos-main/tools”的方案。

步骤 0：实验环境

本实验采用 Docker 容器方案。整体流程分为五步：在宿主机安装 Docker、构建 TinyOS 专用镜像、进入容器、配置环境变量、完成 Blink 仿真程序编译。

- 宿主机：Ubuntu 系统
- 容器基础镜像：ubuntu:16.04
- 目标实验：在 TinyOS 的 Blink 示例目录中执行 `make micaz sim`

说明：仓库 README 的概述提到“Ubuntu 22.04 容器镜像”，但 Dockerfile 实际使用的是 ubuntu:16.04，因此实验时应以 Dockerfile 为准。

步骤 1：在宿主机上安装 Docker Engine

先在宿主机上配置 Docker 官方 apt 仓库，再安装 Docker Engine 及相关插件。依次执行以下命令：

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl

sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

安装完成后，将当前用户加入 docker 组，并检查 Docker 是否可用：

```
sudo usermod -aG docker $USER
newgrp docker
docker version
```

若 `docker version` 能正常输出客户端和服务端版本信息，则说明 Docker 安装成功。

步骤 2：准备 TinyOS 工作目录

在宿主机上创建一个专门的工作目录：

```
mkdir -p ~/tinyos-sim
cd ~/tinyos-sim
```

后续的 Dockerfile、镜像构建和挂载目录都放在这里，便于管理。

步骤 3：创建 Dockerfile

在 `~/tinyos-sim` 目录下新建一个名为 `Dockerfile` 的文件，内容如下：

```
FROM ubuntu:16.04

ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    stow \
    automake \
    autoconf \
    libtool \
    libc6-dev \
    git \
    graphviz \
    python \
    python-serial \
    python2.7-dev \
    python3 \
    python3-serial \
    bison \
    flex \
    gperf \
    emacs-nox \
    default-jdk \
    ca-certificates \
    make \
    gcc \
    g++ \
    wget \
    autoconf \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /opt

# 1) build nesc from source
RUN git clone https://github.com/tinyos/nesc.git && \
```

```

    cd nesc && \
    ./Bootstrap && \
    ./configure && \
    make -j1 && \
    make install

# 2) clone TinyOS main development tree
RUN git clone https://github.com/tinyos/tinyos-main.git /opt/tinyos-main

# 3) build tinyc tools from source
RUN cd /opt/tinyos-main/tools && \
    ./Bootstrap && \
    cd /opt/tinyos-main/tools/tinyos/c/coap && \
    autoreconf -fi && \
    cd /opt/tinyos-main/tools && \
    ./configure && \
    make -j1 && \
    make install

WORKDIR /work
CMD ["/bin/bash"]

```

这份 Dockerfile 的作用是：安装 TinyOS 构建依赖、从源码编译 nesc、拉取 tinycos-main，并在镜像构建阶段完成 tinycos-main/tools 的源码编译。

步骤 4：构建 Docker 镜像

确认当前目录为 ~/tinycos-sim 后，执行：

```

cd ~/tinycos-sim
docker build --no-cache -t tinycos-sim-src16 .

```

说明：--no-cache 表示关闭缓存，避免上一次失败构建的中间层影响本次构建结果。

构建成功后，可用以下命令检查镜像：

```
docker images
```

如果输出中包含 tinycos-sim-src16，则镜像构建成功。

步骤 5：运行容器

先创建一个用于挂载的宿主机目录：

```
mkdir -p ~/tinycos-sim/work
```

方式一：测试模式。退出容器后自动删除：

```

docker run -it --rm \
    --name tinycos-sim \
    -v ~/tinycos-sim/work:/work \
    tinycos-sim-src16

```

方式二：长期使用模式。退出后保留容器：

```
docker run -it \  
  --name tinyos-sim \  
  -v ~/tinyos-sim/work:/work \  
  tinyos-sim-src16
```

容器管理常用命令如下：

```
docker start -ai tinyos-sim  
docker ps -a  
docker stop tinyos-sim  
docker rm tinyos-sim
```

挂载目录的作用是：容器内 /work 下的文件会保存在宿主机的 ~/tinyos-sim/work 中，即使容器删除，工作文件也不会丢失。

步骤 6：容器内配置环境变量

进入容器后，执行以下命令：

```
cat >> ~/.bashrc <<'EOF'  
export TINYOS_ROOT_DIR=/opt/tinyos-main  
export TR=$TINYOS_ROOT_DIR  
export CLASSPATH=.:$TR/support/sdk/java/tinyos.jar  
export PYTHONPATH=$TR/support/sdk/python:$PYTHONPATH  
EOF  
  
source ~/.bashrc
```

配置完成后，可用以下命令检查：

```
echo $TINYOS_ROOT_DIR  
echo $CLASSPATH  
echo $PYTHONPATH
```

如果输出中包含 /opt/tinyos-main，则说明环境变量配置成功。

步骤 7：测试仿真程序

进入 Blink 示例目录：

```
cd /opt/tinyos-main/apps/Blink
```

执行仿真编译命令：

```
make micaz sim
```

说明：micaz 表示目标平台为 MicaZ，sim 表示仿真模式。该命令会将 Blink 示例编译为 TOSSIM 仿真版本。

如果终端最后出现类似 “Successfully built micaz TOSSIM library.” 的提示，则说明实验成功。

步骤 8：实验完成后的结果说明

当 `make micaz sim` 成功执行后，说明以下环节均已打通：

- 宿主机 Docker 可用
- Docker 镜像构建成功
- 容器内 nesc 已完成源码编译
- tinyos-main/tools 已完成源码编译
- Blink 仿真程序已成功构建为 Micaz 平台的 TOSSIM 仿真版本